



BÀI 02: CÁC THÀNH PHẦN CỦA JDBC

- ❑ Các thuộc tính và phương thức của ResultSet(tiếp)
- ❑ Các phương thức của Statement
- ❑ Sử dụng PreparedStatement
- ❑ Xây dựng các phương thức các thao tác CRUD (Create, Read, Update, Delete) .
- ❑ Bài tập



CÁC THUỘC TÍNH VÀ PHƯƠNG THỨC CỦA RESULTSET

- ❑ Đối tượng ResultSet được tạo ra bằng cách sử dụng phương thức `executeQuery()` của Statement để đọc dữ liệu từ cơ sở dữ liệu.
- ❑ Hoặc đối tượng ResultSet cũng có thể được tạo ra từ phương thức `getResultSet()` của đối tượng Statement sau khi đối tượng Statement thực thi câu lệnh truy vấn select thông qua phương thức `execute(String sql)`

❑ VD:

```
stm.execute("SELECT * FROM employees");  
ResultSet rs = stm.getResultSet();
```

- ❑ Dữ liệu đọc được sẽ được lưu trong ResultSet.
- ❑ Có thể di chuyển tới lui trong ResultSet để lấy dữ liệu ra.



CÁC THUỘC TÍNH VÀ PHƯƠNG THỨC CỦA RESULTSET

❑ VD:Phương thức ResultSet

```
18 String sql1 = "select * from sinhvien";
19 ResultSet rs = stm.executeQuery(sql1);
20 while (rs.next()) {
21     System.out.print(rs.getInt(1) + "-");
22     System.out.print(rs.getString("diachi") + "-");
23     System.out.print(rs.getString("hoten") + "-");
24     System.out.print(rs.getString(4) + "-");
25     System.out.println(rs.getInt(5));
26     //System.out.println("");
27 }
```



CÁC PHƯƠNG THỨC CỦA RESULTSET

- ❑ `next()`: di chuyển con trỏ đến dòng kế, trả về `true` nếu có dòng kế tiếp, `false` nếu đến cuối `ResultSet`
- ❑ `previous()`: di chuyển con trỏ đến dòng trước
- ❑ `first()`: di chuyển con trỏ đến dòng đầu tiên
- ❑ `last()`: di chuyển con trỏ đến dòng cuối cùng
- ❑ `beforeFirst()`: di chuyển con trỏ đến vị trí trước dòng đầu tiên
- ❑ `afterLast()`: di chuyển con trỏ đến sau dòng cuối cùng
- ❑ `relative(int rows)`: di chuyển con trỏ tương đối với vị trí hiện tại của nó với số dòng là `rows`
- ❑ `absolute(int row)`: di chuyển con trỏ đến dòng thứ `row`



CÁC THUỘC TÍNH CỦA RESULTSET

❑ TYPE_FORWARD_ONLY

Con trỏ của ResultSet kiểu này chỉ được di chuyển theo một hướng từ đầu đến cuối

❑ TYPE_SCROLL_INSENSITIVE

Con trỏ có thể di chuyển tới lui tương đối với vị trí hiện tại của nó, và cũng có thể di chuyển đến một vị trí cụ thể, không bị ảnh hưởng nếu kết quả được thay đổi ở nơi khác



CÁC THUỘC TÍNH CỦA RESULTSET

Các thuộc tính xác định chế độ hoạt động đồng thời của ResultSet:

- ❑ **CONCUR_READ_ONLY**

Xác định chế độ hoạt động đồng thời, kết quả lưu trong đối tượng ResultSet không được thay đổi

- ❑ **CONCUR_UPDATABLE**

Xác nhận chế độ hoạt động đồng thời, kết quả lưu trong đối tượng ResultSet được thay đổi



CHỈNH SỬA DỮ LIỆU BẰNG RESULTSET

- ❑ Dữ liệu của một bảng trong CSDL có thể được chỉnh sửa bằng cách sử dụng câu lệnh SQL UPDATE
- ❑ Ta cũng có thể chỉnh sửa dữ liệu trên các hàng của một bảng từ ResultSet

Để làm được điều này, ResultSet phải được đặt ở chế độ **CONCUR_UPDATABLE**

Sử dụng các phương thức `updateXXX()` để chỉnh sửa dữ liệu trên ResultSetm (XXX là kiểu dữ liệu của cột cần được sửa)



CHỈNH SỬA DỮ LIỆU BẰNG RESULTSET

□ VD:

```
public void updateTable() {  
  
    String selectQuery = "SELECT * FROM PRICELIST;";  
    if (conn != null) {  
        try {  
            Statement st =  
                conn.createStatement(ResultSet.TYPE_SCROLL_SENSITIVE,  
                    ResultSet.CONCUR_UPDATABLE);  
            ResultSet rs = st.executeQuery(selectQuery);  
            rs.next();  
            rs.updateDouble("PRICE", 1.5);  
            rs.updateRow();  
            st.close();  
        } catch (SQLException e) {  
            e.printStackTrace();  
        }  
    }  
}
```



SỬ DỤNG PREPAREDSTATEMENT

- ❑ Sử dụng phương thức executeUpdate() của Statement để cập nhật dữ liệu qua JDBC
- ❑ executeUpdate() trả về số dòng bị ảnh hưởng sau khi câu lệnh được thực thi

```
private void insertRow(int _id, String _hoTen, String _diaChi, String _tenLop,
    String ns) {
    String insertQuery = "insert into sinhvien values ('" + _id + "','" + _hoTe
        + _diaChi + "','" + _tenLop + "','" + ns + "');"
    //+_diaChi+"','"+_tenLop+"','2005/01/01');"
    System.out.println(insertQuery);
    if (this.connection != null) {
        try {
            Statement st = connection.createStatement();
            int n = st.executeUpdate(insertQuery);
            System.out.println("insert ok");
            st.close();
        } catch (SQLException ex) {
            ex.printStackTrace();
        }
    }
}

public static void main(String args[]) {
    ConnectionDatabase kn = new ConnectionDatabase();
    kn.connect();
    // kn.showData(kn.getData());
    // kn.createTable();
    kn.insertRow(7, "Hoang van Trung", "Thai Nguyen", "K13B", "2008/01/01");
    kn.closeConnec();
}
```



SỬ DỤNG PREPARED STATEMENT

❑ Sử dụng Statement khi:

- ✓ Truy vấn của bạn đơn giản và chỉ thực hiện một lần, không cần tái sử dụng.
- ✓ Không cần xử lý các tham số phức tạp hoặc các biến đầu vào từ người dùng.
- ✓ Làm việc với các truy vấn không yêu cầu sự an toàn cao và không có rủi ro bị SQL Injection, chẳng hạn như các truy vấn nội bộ không có dữ liệu từ người dùng.



SỬ DỤNG PREPAREDSTATEMENT

❑ Sử dụng PreparedStatement khi:

- ✓ Cần thực hiện truy vấn nhiều lần với các tham số khác nhau, ví dụ: truy vấn lặp lại với các giá trị khác nhau trong vòng lặp.
- ✓ Các truy vấn phức tạp, yêu cầu nhiều điều kiện hoặc cần tái sử dụng truy vấn.
- ✓ Ngăn ngừa SQL Injection, đặc biệt khi có dữ liệu đầu vào từ người dùng.
- ✓ Cải thiện hiệu suất của ứng dụng bằng cách giảm chi phí biên dịch lại câu lệnh SQL.



SỬ DỤNG PREPAREDSTATEMENT

- ❑ VD: Sử dụng PreparedStatement khi thực hiện thêm một bản ghi:

```
private void insertRowPreparedStatement(int _id, String _hoTen, String _diaChi,
    String _tenLop, String _ns) {
    String insertQuery = "INSERT INTO SINHVIEN VALUES (?, ?, ?, ?, ?)";
    if (this.connection != null) {
        try { //connection.setAutoCommit(false);
            PreparedStatement pst = connection.prepareStatement(insertQuery);
            pst.setInt(1, _id);
            pst.setString(2, _hoTen);
            pst.setString(3, _diaChi);
            pst.setString(4, _tenLop);
            pst.setString(5, _ns);
            System.out.println(pst.toString());
            int n = pst.executeUpdate();
            System.out.println("insert ok");
            pst.close();
            // connection.commit();
            //connection.setAutoCommit(true);
        } catch (SQLException ex) {
            ex.printStackTrace();
        }
    }
}
```



XÂY DỰNG CÁC PHƯƠNG THỨC CÁC THAO TÁC CRUD (CREATE, READ, UPDATE, DELETE) .

- ❑ Lớp biểu diễn thực thể **SinhVien** tương ứng với bảng sinhvien
 - ❑ Lớp SinhVienDAO với các phương thức:
 - ✓ **boolean addSinhVien(SinhVien sv)**: Thêm một bản ghi SinhVien vào cơ sở dữ liệu.
 - ✓ **SinhVien getByld(int id)**: Truy vấn một sinh viên cụ thể theo id.
 - ✓ **List<SinhVien> getAll()**: Truy vấn và trả về danh sách tất cả sinh viên trong bảng SinhVien.
 - ✓ **boolean update(SinhVien sv)**: Cập nhật thông tin của một sinh viên trong cơ sở dữ liệu.
 - ✓ **boolean delete(int id)**: Xóa một sinh viên khỏi cơ sở dữ liệu dựa trên id.
 - ✓ ...
- Các lớp này tạo thành mô hình DAO cho thực thể SinhVien, giúp cho việc tương tác với cơ sở dữ liệu trở nên đơn giản, rõ ràng, và dễ bảo trì hơn.



XÂY DỰNG LỚP SINHVIEN

```
3 public class SinhVien {
4     private int id;
5     private String hoTen;
6     private String diaChi;
7     private String tenLop;
8     private String namSinh;
9     public SinhVien() {}
10    public SinhVien(int id, String hoTen, String diaChi, String tenLop, String namSinh) {
11        this.id = id;
12        this.hoTen = hoTen;
13        this.diaChi = diaChi;
14        this.tenLop = tenLop;
15        this.namSinh = namSinh;
16    }
17    // Getters và Setters
18    public int getId() {
19        return id;
20    }
21
22    public void setId(int id) {
23        this.id = id;
24    }
25
26    public String getHoTen() {
27        return hoTen;
28    }
```



PHƯƠNG THỨC ĐÓNG TÀI NGUYÊN KẾT NỐI

```
126 private void closeResources(Connection connection, Statement statement, ResultSet rs) {  
127     try {  
128         if (rs != null) {  
129             rs.close();  
130         }  
131         if (statement != null) {  
132             statement.close();  
133         }  
134         if (connection != null) {  
135             connection.close();  
136         }  
137     } catch (SQLException e) {  
138         e.printStackTrace();  
139     }  
140 }  
141 }
```



PHƯƠNG THỨC ADDSINHVIEN THÊM MỘT BẢN GHI SINHVIEN VÀO CƠ SỞ DỮ LIỆU.

```
9 public boolean add(SinhVien sv) {
10     Connection connection = null;
11     PreparedStatement Pstatement = null;
12 String sql = "INSERT INTO SinhVien (hoTen, diaChi, tenLop, namSinh) VALUES (?, ?, ?, ?)";
13
14     try {
15         connection = JDBCUtil.getConnection();
16         Pstatement = connection.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS);
17
18         Pstatement.setString(1, sv.getHoTen());
19         Pstatement.setString(2, sv.getDiaChi());
20         Pstatement.setString(3, sv.getTenLop());
21         Pstatement.setString(4, sv.getNamSinh());
22
23         int affectedRows = Pstatement.executeUpdate();
24         return affectedRows > 0;
25     } catch (SQLException e) {
26         e.printStackTrace();
27     } finally {
28         closeResources(connection, Pstatement, null);
29     }
30     return false;
31 }
32
```



TRUY VẤN VÀ TRẢ VỀ DANH SÁCH TẤT CẢ SINH VIÊN TRONG BẢNG SINHVIEN.

```
57 public List<SinhVien> getAll() {
58     Connection connection = null;
59     Statement statement = null;
60     ResultSet rs = null;
61     List<SinhVien> list = new ArrayList<SinhVien>();
62     String sql = "SELECT * FROM SinhVien";
63     try {
64         connection = JDBCUtil.getConnection();
65         statement = connection.createStatement();
66         rs = statement.executeQuery(sql);
67
68         while (rs.next()) {
69             SinhVien sv = new SinhVien(rs.getInt("id"), rs.getString("hoTen"),
70             rs.getString("diaChi"), rs.getString("tenLop"), rs.getString("namSinh"));
71             list.add(sv);
72         }
73     } catch (SQLException e) {
74         e.printStackTrace();
75     } finally {
76         closeResources(connection, statement, rs);
77     }
78     return list;
79 }
80
```



CẬP NHẬT THÔNG TIN CỦA MỘT SINH VIÊN TRONG CƠ SỞ DỮ LIỆU

```
81 public boolean update(SinhVien sv) {
82     Connection connection = null;
83     PreparedStatement statement = null;
84     String sql = "UPDATE SinhVien SET hoTen = ?, diaChi = ?, tenLop = ?, "
85                 + "namSinh = ? WHERE id = ?";
86     try {
87         connection = JDBCUtil.getConnection();
88         statement = connection.prepareStatement(sql);
89
90         statement.setString(1, sv.getHoTen());
91         statement.setString(2, sv.getDiaChi());
92         statement.setString(3, sv.getTenLop());
93         statement.setString(4, sv.getNamSinh());
94         statement.setInt(5, sv.getId());
95
96         int affectedRows = statement.executeUpdate();
97         return affectedRows > 0;
98     } catch (SQLException e) {
99         e.printStackTrace();
100    } finally {
101        closeResources(connection, statement, null);
102    }
103    return false;
104 }
```



PHƯƠNG THỨC XÓA MỘT BẢN GHI SINHVIEN TRONG SỔ DỮ LIỆU.

```
106 public boolean delete(int id) {  
107     Connection connection = null;  
108     PreparedStatement statement = null;  
109     String sql = "DELETE FROM SinhVien WHERE id = ?";  
110  
111     try {  
112         connection = JDBCUtil.getConnection();  
113         statement = connection.prepareStatement(sql);  
114         statement.setInt(1, id);  
115  
116         int affectedRows = statement.executeUpdate();  
117         return affectedRows > 0;  
118     } catch (SQLException e) {  
119         e.printStackTrace();  
120     } finally {  
121         closeResources(connection, statement, null);  
122     }  
123     return false;  
124 }  
125
```



CÁC THÀNH PHẦN CỦA JDBC

❑ Bài tập thực hành

Bài 1: Tạo một lớp DAO đơn giản để thêm thông tin người dùng vào cơ sở dữ liệu.

Yêu cầu:

- ✓ Tạo một bảng User trong cơ sở dữ liệu với cấu trúc như sau:
 - id (int, tự động sinh)
 - username (varchar, không null)
 - password (varchar, không null)
 - email (varchar)
- ✓ Tạo lớp User với các thuộc tính tương ứng: id, username, password, và email.
- ✓ Tạo lớp UserDao để quản lý các hoạt động truy vấn cơ sở dữ liệu cho bảng User.
 - Viết phương thức addUser(User user) để thêm người dùng mới vào bảng User bằng cách sử dụng PreparedStatement.
 - Phương thức này sẽ trả về true nếu thêm thành công, false nếu có lỗi xảy ra.

Gợi ý:

- ✓ Sử dụng câu lệnh SQL INSERT INTO User (username, password, email) VALUES (?, ?, ?);.
- ✓ Trong addUser, sử dụng PreparedStatement để đặt các tham số và thực thi truy vấn.



CÁC THÀNH PHẦN CỦA JDBC

❑ Bài tập thực hành

Bài 2: Mở rộng lớp UserDao với các phương thức tìm kiếm và cập nhật thông tin người dùng.

Yêu cầu:

- ✓ Trong lớp UserDao, tạo các phương thức sau:
 - findUserByUsername(String username): Tìm và trả về đối tượng User dựa trên username.
 - updateUserEmail(int id, String newEmail): Cập nhật địa chỉ email của người dùng với id cụ thể.
- ✓ Sử dụng PreparedStatement để viết truy vấn cho các phương thức:
 - findUserByUsername sử dụng câu lệnh `SELECT * FROM User WHERE username = ?;`.
 - updateUserEmail sử dụng câu lệnh `UPDATE User SET email = ? WHERE id = ?;`.
- ✓ Đảm bảo mỗi phương thức đóng tài nguyên Connection, PreparedStatement, và ResultSet (nếu có) sau khi thực thi xong.

Yêu cầu bổ sung:

- ✓ Kiểm tra kết quả trả về cho findUserByUsername, nếu không tìm thấy, phương thức sẽ trả về null.
- ✓ updateUserEmail trả về true nếu cập nhật thành công, false nếu không thành công.



TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN

Thank you!

THÁI NGUYÊN, THÁNG NĂM